

Chatbot médical RAG

Un assistant informatif basé sur des documents officiels (HAS / INCa)

SAE BUT3 Informatique — S6 — Mr FAYE & Mme AZZAG

Yousra · Imane · Yannis · Grace

Sommaire

Chatbot médical RAG · 15 minutes · 4 intervenants

01

Contexte & Architecture

Grace · slides 3 à 7

02

Traitement des PDFs & Pipeline RAG

Yousra · slides 8 à 13

03

Comparaison encodeurs & DuckDuckGo

Imane · slides 14 à 18

04

Hermes, Déploiement & Démo

Yannis · slides 19 à 26

05

Conclusion & Questions

Tous · slides 27 & 28

PARTIE 1

Contexte & Architecture

Grace

Le besoin

01

Un patient veut une info médicale rapide et fiable

02

Un LLM seul peut halluciner — inventer des informations médicales fausses

03

Objectif : un chatbot qui répond à partir de sources officielles, pas de sa mémoire générale

Domaine choisi : informatif uniquement — pas de diagnostic

Pourquoi un LLM "tout fait" et pas créé from scratch ?

L'enjeu n'est pas de réinventer un LLM, mais de savoir l'orchestrer.

*Analogie : on ne réinvente pas le moteur d'une voiture,
on construit le châssis, la direction, le tableau de bord.*

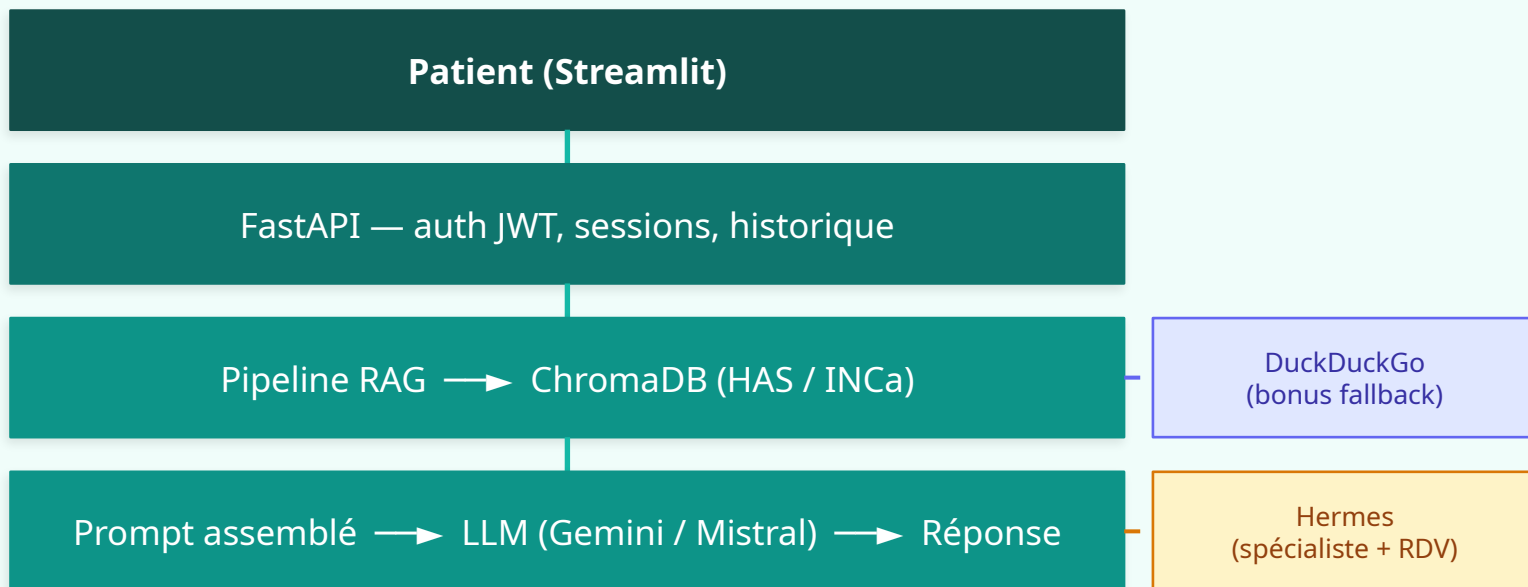
Le vrai travail réalisé :

- Pipeline RAG : extraction, chunking, indexation, recherche vectorielle
- Backend FastAPI, authentification JWT, gestion des sessions et de l'historique
- Prompt engineering : contexte, historique, domaine restreint
- Comparaison d'encodeurs et de LLMs

Stack technique

Couche	Technologie
Frontend	Streamlit
Backend	FastAPI + SQLAlchemy
Auth	JWT (bcrypt + PyJOSE)
RAG	LangChain + ChromaDB
Embeddings	all-MiniLM-L6-v2 (comparé à 2 autres)
LLM	Gemini 2.5 Flash (prod) / Mistral via OpenRouter (dev)
Recherche web	DuckDuckGo (ddgs) — bonus
Base de données	SQLite

Architecture générale



PARTIE 2

Traitement des PDF & Pipeline RAG

Yousra

Étape 1 — Indexation des documents (une seule fois)

5 PDF officiels (HAS, INCa) : diabète · Alzheimer · cancer du poumon

971
chunks

01

Extraction — PyPDFLoader lit le texte page par page

02

Découpage — RecursiveCharacterTextSplitter — 500 caractères, chevauchement 50

03

Encodage — Chaque chunk → vecteur de 384 nombres (MiniLM)

04

Stockage — ChromaDB conserve le texte + vecteur liés par un identifiant

Pourquoi un chevauchement de 50 caractères ?

Sans chevauchement, une phrase clé peut être coupée en deux :

```
Chunk 1 : "...repose sur la metfor"  
Chunk 2 :  
"mine en première intention..."
```

Avec un chevauchement de 50 caractères :

Le chunk suivant recule de 50 caractères avant de repartir → la phrase complète apparaît dans au moins un chunk.

La phrase "repose sur la metformine" se retrouve intacte dans au moins un chunk.

Le rôle de MiniLM

MiniLM transforme n'importe quel texte en un vecteur de 384 nombres qui capture son sens — pas juste les mots.

500

taille du chunk
(caractères)

384

taille du vecteur
(toujours fixe)

971

chunks indexés
au total

"fièvre" et "température élevée" → vecteurs proches, même avec des mots différents.

Étape 2 — Répondre à une question (RAG en action)

- 1 Question du patient → convertie en vecteur (même MiniLM)
- 2 ChromaDB cherche les 4 chunks les plus proches par sens
- 3 Mauvais score → hors périmètre des PDF → bascule sur la recherche web
- 4 Contexte (chunks + historique) assemblé dans un prompt
- 5 Le LLM lit ce prompt et rédige la réponse finale

La similarité : comment on compare deux vecteurs

Score = distance L2 (ChromaDB) — plus le score est bas, plus le chunk est pertinent (inverse d'un score de similarité classique)

```
hits = vectorstore.similarity_search_with_score(question, k=4)
needs_web_search = min(score for _, score in hits) > RELEVANCE_THRESHOLD
```

RELEVANCE_THRESHOLD = 0.9 : au-delà, aucun chunk n'est assez pertinent
→ on bascule sur le web plutôt que de répondre "à côté"

Distance L2 = racine de $\sum(a_i - b_i)^2$ sur 384 dimensions — mesure géométrique entre deux vecteurs.

PARTIE 3

Comparaison des encodeurs & Bonus DuckDuckGo

Imane

Pourquoi comparer plusieurs encodeurs ?

Le choix de l'encodeur change la **qualité de la recherche sémantique**
— exigence explicite de la SAE : "tester et comparer plusieurs encodeurs"

Encodeur	Dim.	Spécialité
all-MiniLM-L6-v2	384	Généraliste multilingue, très léger
sentence-camembert-base	768	Spécialisé français
paraphrase-multilingual-mpnet-base-v2	768	Multilingue haute qualité

3 modèles testés sur les mêmes 5 PDF, indexés séparément.

Résultats de la comparaison

Encodeur	Vitesse (971 chunks)	Qualité observée
MiniLM (retenu)	71s	Bonne, parfois distraite par du bruit (citations)
CamemBERT	270s	Capte mieux le sens médical en français
mpnet multilingue	244s	Le plus pertinent des 3 sur nos tests

Décision : MiniLM gardé en prod — rapidité d'indexation (4× plus rapide que CamemBERT/mpnet)

Les 2 autres documentés comme alternatives plus précises mais plus lentes.

→ Répond à l'exigence "comparer" sans sacrifier les contraintes de déploiement.

Bonus — Recherche web DuckDuckGo

Quand le RAG ne couvre pas la question (score > seuil), on enrichit la réponse avec une recherche web — pour ne pas répondre "à côté".

```
def web_search(question, max_results=3):  
    query = reformulate_search_query(question)  
    results = DDGS().text(query, max_results=max_results)  
  
    return "\n\n".join(f"{r['title']} : {r['body']} (source: {r['href']})" for r in  
    results)
```

Web = filet de sécurité, jamais une source par défaut.

Le LLM signale explicitement quand une info vient du web, pas des PDF.

Bug rencontré et corrigé (DuckDuckGo)

Symptôme

"j'ai de la fièvre" interprété comme une demande de traduction → résultats Reverso / Collins, pas d'info médicale.

Cause

La phrase patient brute ressemble à une phrase de manuel de langue.

Fix

reformulate_search_query() — un appel LLM reformule la phrase en requête médicale propre avant DuckDuckGo.

"j'ai de la fièvre" → "fièvre symptômes causes traitement" → résultats médicaux réels ✓

PARTIE 4

Hermes, Déploiement & Démo

Yannis

Bonus — Hermes, orchestrateur multi-agents

Après une réponse du chatbot, le patient peut demander à voir un spécialiste.

1

recommend_specialist

Quel type de médecin consulter — analyse la conversation complète

2

list_available_doctors

Liste les créneaux disponibles (5 médecins de démo)

3

book_appointment

Réserve le RDV et le retire des disponibilités du médecin

Modèle : Hermes 405B via OpenRouter, avec fallback automatique vers Llama 70B / GPT-OSS.

Bonus++ — Hermes côté praticien (Telegram)

Une fois le RDV pris, le praticien reçoit + peut interagir via son groupe Telegram.

1



Notif sortante

Nouvelle réservation → message automatique dans le groupe du Dr.

2



Commandes entrantes

/aide /rdv_aujourd'hui /rdv_demain /rdv + langage naturel via Gemini

3



Cloisonnement

1 groupe Telegram = 1 médecin (mapping Doctor.telegram_chat_id)

Démo SAE : 2 groupes (Dr. Lefebvre, Dr. Benyahia). Infra prête pour N praticiens.

Sécurité & limites assumées

Pas de diagnostic

Le chatbot n'est pas un médecin — rappel explicite dans chaque prompt envoyé au LLM.

Traçabilité des sources

Badge "documents officiels" vs "web" sur chaque réponse — l'utilisateur sait toujours d'où vient l'info.

Limites de déploiement assumées

Périmètre limité à 3 pathologies. Quota LLM free tier ~1000 req/jour. ChromaDB et SQLite sur volume Docker persistant — pas de perte au redéploiement.

Déploiement en production

<https://sae.amanawebagency.com>

Caddy — HTTPS automatique — Let's Encrypt

Streamlit — Container Docker — Interface chat (/app/)

FastAPI — Container Docker — Réseau interne (non exposé)

ChromaDB + SQLite — Volume persistant Docker

Intégration site + chatbot

L'utilisateur n'arrive jamais sur Streamlit nu

Landing / — Site MediGuide (HTML/CSS) — Présentation, sources, disclaimer

Page /consulter — iframe du chatbot dans le header MediGuide

Chatbot /app/ — Streamlit harmonisé (même palette teal, font Inter)

Cohérence visuelle — Pages login + chat reprennent le style MediGuide

Site MediGuide → bouton « Consulter » → /consulter (iframe /app/) → chatbot dans le contexte de la marque

Démo en direct

1

Question dans le périmètre (diabète)

→ réponse basée sur HAS, badge "documents officiels"

2

Question hors périmètre (grippe)

→ bascule web, sources citées, badge "web"

3

Hermes

→ recommandation spécialiste + prise de RDV

Polish UX — les détails qui comptent



Cohérence visuelle

Palette teal & font Inter partagées entre site et chatbot



Badges de sources

Chaque réponse affiche « Documents officiels HAS/INCa » ou « Recherche web »



Mes rendez-vous

Section permanente dans la sidebar — historique des RDV pris, toujours retrouvable



Auto-rename des sessions

Le titre devient les 60 premiers caractères du 1er message — sidebar lisible

Bilan des compétences mobilisées

Full Stack

FastAPI + Streamlit + JWT + SQLAlchemy

IA / LLM

RAG, embeddings, prompt engineering, comparaison de modèles

Esprit critique

Limites du RAG, seuil de pertinence, sécurité (pas de diagnostic)

DevOps

Docker, reverse proxy Caddy, déploiement réel en production

Merci

Questions ?

SAE BUT3 Informatique — S6 — Mr FAYE & Mme AZZAO

Yousra · Imane · Yannis · Grace

<https://sae.amanawebagency.com>



Scan → MediGuide